

Problem 1

Problem Statement: Student Marks Analyzer (LEVEL-1)

Scenario:

You are developing a small utility for a teacher to analyze student marks stored in an array.

Requirements

- Store student marks in an array.
- Calculate total and average marks using array methods.
- Display pass/fail result based on average.
- Use let/const appropriately.
- Use arrow functions for calculations.
- Display output using template literals.

Technical Constraints

- Must use array methods like reduce() and map().
- Use only ES6+ syntax.
- No external libraries.
- All logic must be inside modular JavaScript file.

Learning Outcome

- Understand array processing.
- Practice ES6 features.
- Learn modular coding structure.

Source Code:

File Name: Index.html

```
p1 > index.html > ...
1  <!--JS Day7 Hands_on_Problem_Statement1_Harshitha Kamatam-->
2  <!DOCTYPE html>
3  <html>
4  <head>
5      <meta charset="UTF-8">
6      <meta name="viewport" content="width=device-width, initial-scale=1.0">
7      <title>Student Marks Analyzer</title>
8
9      <style>
10         body{
11             text-align: center;
12             background-color: #beige;
13         }
14     </style>
15 </head>
16 <body>
17     <h2>Student Marks Analyzer</h2>
18     <div id="output"></div>
19     <script src="script.js"></script>
20 </body>
21 </html>
```

File Name: script.html

```
p1 > js Scriptjs > ...
1  //JS Day7 Hands_on_Problem_Statement1_Harshitha Kamatam
2  const marks = [75, 80, 65, 90, 55];
3  const calculateTotal = (arr) => arr.reduce((sum, mark) => sum + mark, 0);
4
5  const calculateAverage = (arr) => {
6      const total = calculateTotal(arr);
7      return total / arr.length;
8  };
9
10 const getResult = (average) => (average >= 40 ? "Pass" : "Fail");
11
12 const formattedMarks = marks.map((mark, index) => `Subject ${index + 1}: ${mark}`);
13
14 const totalMarks = calculateTotal(marks);
15 const averageMarks = calculateAverage(marks);
16 const result = getResult(averageMarks);
17
18 console.log(`
19 Student Marks Report
20 -----
21 ${formattedMarks.join("\n")}
22
23 Total Marks : ${totalMarks}
24 Average Marks : ${averageMarks.toFixed(2)}
25 Result : ${result}
26 `);
27
28 document.getElementById("output").innerHTML = `
29 <p>Student Marks Report</p>
30 <p>-----</p>
31 <p>${formattedMarks.join(", ")}</p>
32 <p>Total Marks : ${totalMarks}</p>
33 <p>Average Marks : ${averageMarks.toFixed(2)}</p>
34 <p>Result : ${result}</p>
35 `;
```

Output:

Output File: index.html

Student Marks Analyzer

Student Marks Report

Subject 1: 75, Subject 2: 80, Subject 3: 65, Subject 4: 90, Subject 5: 55

Total Marks : 365

Average Marks : 73.00

Result : Pass

Explanation:

This code creates a **Student Marks Analyzer** that stores marks in an array and calculates the total and average using JavaScript functions. It also checks if the student passed or failed based on the average. The output shows each subject mark, the total marks, the average marks, and the final result (Pass/Fail) displayed on the webpage.

Problem 2

Problem Statement: Product Cart Summary (LEVEL-1)

Scenario:

Build a simple shopping cart summary system.

Requirements

- Store product objects (name, price, quantity) in an array.
- Calculate total cart value.
- Display formatted invoice using template literals.
- Use arrow functions.
- Export calculation function from a module.

Technical Constraints

- Use map() and reduce().
- Use ES6 modules (export/import).
- No DOM manipulation required.

Learning Outcome

- Work with arrays of objects.
- Understand modules.
- Improve functional programming skills.

Source Code:

File Name: index.html

```
p2 > index.html > ...
1  <!--JS Day7 Hands on Problem Statement2 Harshitha Kamatam-->
2  <!DOCTYPE html>
3  <html>
4  <head>
5      <meta charset="UTF-8">
6      <meta name="viewport" content="width=device-width, initial-scale=1.0">
7      <title>Product Cart Summary</title>
8      <style>
9          body{
10             text-align: center;
11             background-color: #beige;
12         }
13     </style>
14 </head>
15 <body>
16     <h2>product Cart Summary</h2>
17 </body>
18 </html>
```

File Name: script.js

```
p2 > JS scriptjs > ...
1  const products = [
2      { name: "Laptop", price: 43750, quantity: 1 },
3      { name: "Mouse", price: 500, quantity: 1 },
4      { name: "Keyboard", price: 1500, quantity: 1 },
5      { name: "Laptop Bag", price: 1200, quantity: 1 }
6  ];
7
8  const calculateTotal = (items) =>
9      items
10         .map(item => item.price * item.quantity)
11         .reduce((total, value) => total + value, 0);
12
13  const generateInvoice = (items) => {
14      const total = calculateTotal(items);
15      const productDetails = items
16         .map(item =>
17             `${item.name} - ${item.price} x ${item.quantity} = ${item.price * item.quantity}`
18         )
19         .join("\n");
20
21      return `
22  ----- INVOICE -----
23  ${productDetails}
24  Total Amount: ${total}
25  -----
26  `;
27  };
28
29  console.log(generateInvoice(products));
```

Output:

Output File: index.html

Product Cart Summary

Output File: script.js

```
D:\upGrad\Training\Week2\Day2\Problem Statement 2>node script.js
```

```
----- INVOICE -----  
Laptop - ₹43750 x 1 = ₹43750  
Mouse - ₹500 x 1 = ₹500  
Keyboard - ₹1500 x 1 = ₹1500  
Laptop Bag - ₹1200 x 1 = ₹1200  
-----  
Total Amount: ₹46950  
-----
```

Explanation:

This code creates a **Product Cart Summary** using a list of products with price and quantity. JavaScript calculates the total cost using `map()` and `reduce()` and generates a simple invoice. The output shows each product with its price calculation and the **final total amount of ₹46950** printed in the console.

Problem 3

Problem Statement: **Weather Data Fetcher (LEVEL-2)**

Scenario:

Create an application that fetches weather data asynchronously.

Requirements

- Fetch data from a public API using `fetch()`.
- Implement version using Promises.
- Implement version using `async/await`.
- Handle errors properly.
- Display formatted weather report.

Technical Constraints

- Use `async/await`.
- Use template literals.
- Use arrow functions.
- Must include proper error handling using `try/catch`.

Learning Outcome

- Understand Promises and `async/await`.
- Handle asynchronous operations.
- Improve API handling skills.

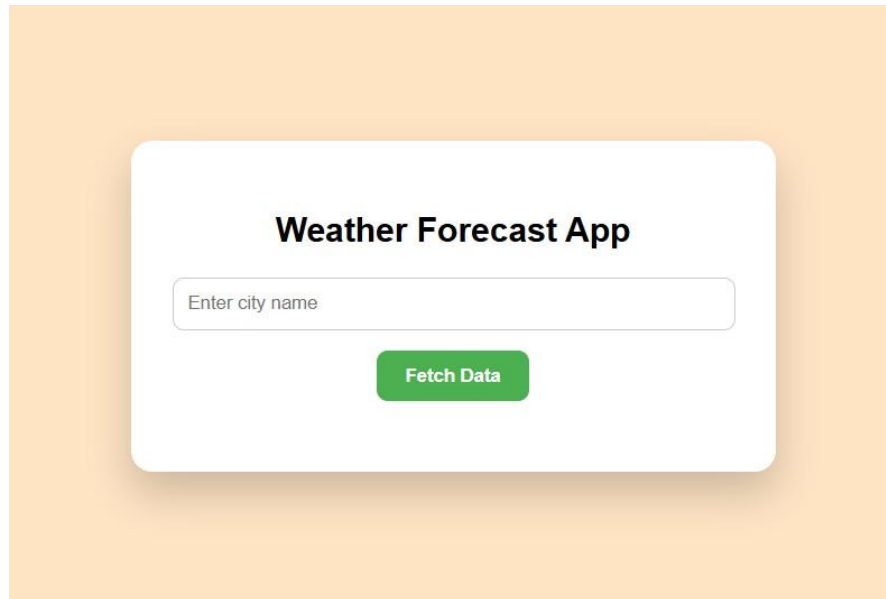
Source Code:

File Name: index.html

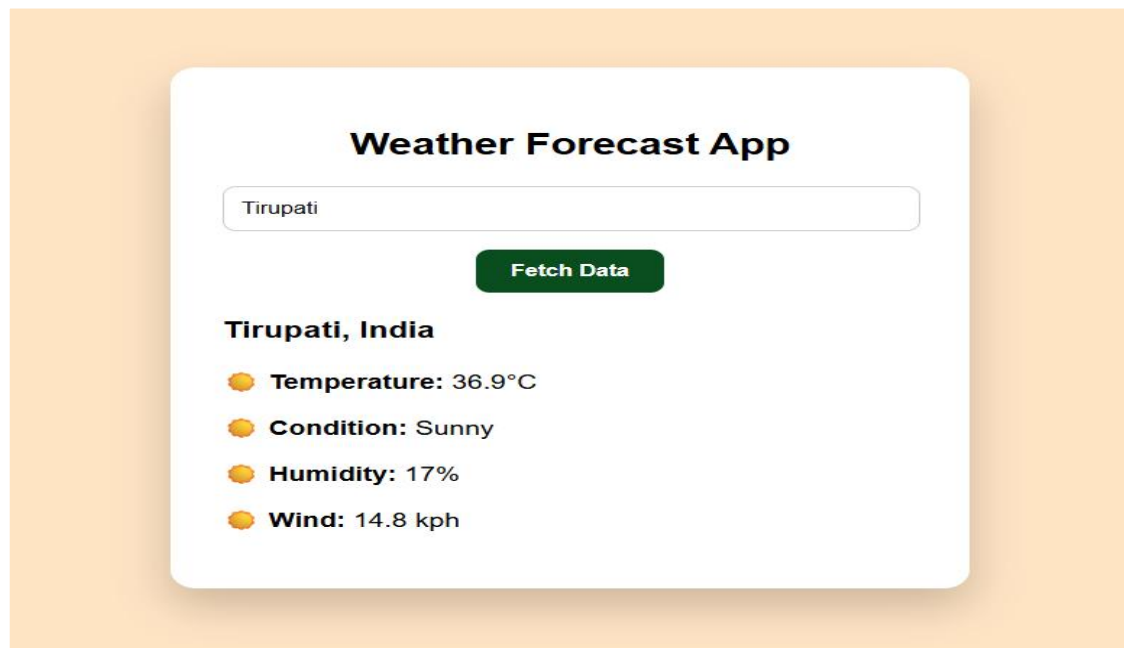
```
p3 > index.html > html > body > script
1 <!--JS Day7 Hands_on_Problem_Statement3_Harshitha_Kamatam-->
2 <!DOCTYPE html>
3 <html lang="en">
4 <head>
5 <meta charset="UTF-8">
6 <meta name="viewport" content="width=device-width, initial-scale=1.0">
7 <title>Weather Fetcher</title>
8 <style>
9 body {
10   font-family: Arial, sans-serif;
11   background: #bisque;
12   display: flex;
13   justify-content: center;
14   align-items: center;
15   height: 100vh;
16   margin: 0;
17 }
18 .container {
19   background: #white;
20   padding: 30px;
21   border-radius: 15px;
22   width: 400px;
23   text-align: center;
24   box-shadow: 0 15px 30px #rgba(0,0,0,0.2);
25 }
26 input {
27   width: 95%;
28   padding: 10px;
29   margin-bottom: 15px;
30   border-radius: 8px;
31   border: 1px solid #ccc;
32 }
33 button {
34   padding: 10px 20px;
35   border: none;
36   border-radius: 8px;
37   cursor: pointer;
38   font-weight: bold;
39   background-color: #0a4a2b;
40   color: #white;
41 }
42 button:hover {
43   opacity: 0.9;
44 }
45 #weatherResult {
46   margin-top: 20px;
47   text-align: left;
48 }
49 </style>
50 </head>
51 <body>
52 <div class="container">
53 <h2>Weather Forecast App</h2>
54 <input type="text" id="cityInput" placeholder="Enter city name">
55 <button id="fetchBtn">Fetch Data</button>
56 <div id="weatherResult"></div>
57 </div>
58
59 <script>
60 const cityInput = document.getElementById("cityInput");
61 const resultDiv = document.getElementById("weatherResult");
62
63 const displayWeather = (data) => {
64   resultDiv.innerHTML = `
65     <h3>${data.location.name}, ${data.location.country}</h3>
66     <p><strong>🌡️ Temperature:</strong> ${data.current.temp_c}°C</p>
67     <p><strong>🌤️ Condition:</strong> ${data.current.condition.text}</p>
68     <p><strong>💧 Humidity:</strong> ${data.current.humidity}%</p>
69     <p><strong>💨 Wind:</strong> ${data.current.wind_kph} kph</p>
70   `;
71 };
72
73 const fetchWeatherWithPromises = (searchCity) => {
74   const url = `https://api.weatherapi.com/v1/forecast.json?key=f567c39b04ac410dbed174730260403&q=${searchCity}&days=7&aqi=yes&alerts=no`;
75   return fetch(url)
76     .then(response => {
77       if (!response.ok) {
78         throw new Error("City Not Found or API Error");
79       }
80       return response.json();
81     });
82 };
83
84 const fetchWeatherAsync = async (searchCity) => {
85   const url = `https://api.weatherapi.com/v1/forecast.json?key=f567c39b04ac410dbed174730260403&q=${searchCity}&days=7&aqi=yes&alerts=no`;
86   try {
87     const response = await fetch(url);
88     if (!response.ok) {
89       throw new Error("City Not Found or API Error");
90     }
91     const data = await response.json();
92     displayWeather(data);
93   } catch (error) {
94     resultDiv.innerHTML = `Error: ${error.message}`;
95   }
96 };
97 document.getElementById("fetchBtn").addEventListener("click", () => {
98   const searchCity = cityInput.value.trim();
99   if (!searchCity) {
100     resultDiv.innerHTML = "Please Enter a City Name.";
101     return;
102   }
103   fetchWeatherAsync(searchCity);
104 });
105 </script>
106 </body>
107 </html>
```


Output:

Output File: index.html



The image shows the initial state of a web application titled "Weather Forecast App". It features a white rounded rectangle centered on an orange background. Inside the rectangle, there is a text input field with the placeholder text "Enter city name" and a green button labeled "Fetch Data".



The image shows the application after a data fetch. The input field now contains the text "Tirupati". The "Fetch Data" button is now dark green. Below the button, the location "Tirupati, India" is displayed. A list of weather details follows, each preceded by a sun icon: "Temperature: 36.9°C", "Condition: Sunny", "Humidity: 17%", and "Wind: 14.8 kph".

Tirupati, India	
	Temperature: 36.9°C
	Condition: Sunny
	Humidity: 17%
	Wind: 14.8 kph

Weather Forecast App

Please Enter a City Name.

Weather Forecast App

Error: City Not Found or API Error

Explanation:

This code creates a **Weather Forecast App** where the user enters a city name and clicks **Fetch Data** to get weather information from a public API. JavaScript uses **fetch() with async/await** to retrieve temperature, condition, humidity, and wind details and display them on the page. If the city is empty or invalid, it shows an **error message** like “Please Enter a City Name” or “City Not Found”.

Problem 4

Problem Statement: **Task Manager with Async Storage (LEVEL-2)**

Scenario:

Develop a task manager where tasks are saved and retrieved asynchronously.

Requirements

- Store tasks in an array.
- Create `addTask()`, `deleteTask()`, and `listTasks()` functions.
- Simulate async storage using `setTimeout` with callbacks.
- Convert callback-based logic into Promises.
- Refactor into `async/await` version.
- Use ES6 modules.

Technical Constraints

- Must demonstrate `callback` → `promise` → `async/await` evolution.
- Use `let/const` properly.
- Use arrow functions.
- Use template literals for display.

Learning Outcome

- Deep understanding of async JavaScript.
- Learn refactoring async code.
- Master modern ES6+ features.

Source Code:

File Name: index.html

```
1  <!--JS Day7_Hands_on_Problem_Statement4_Harshitha_Kamatam-->
2  <!DOCTYPE html>
3  <html>
4  <head>
5    <meta charset="UTF-8">
6    <meta name="viewport" content="width=device-width, initial-scale=1.0">
7    <title>Task Manager</title>
8    <style>
9    body{
10     font-family: Arial;
11     text-align: center;
12     margin-top: 50px;
13     background-color: #007bff;
14   }
15   input{
16     padding:8px;
17     width:200px;
18   }
19   button{
20     padding:8px 12px;
21     margin-left:5px;
22   }
23   li{
24     margin:10px;
25     list-style:none;
26   }
27   .deleteBtn{
28     margin-left:10px;
29     background: #dc3545;
30     color: #fff;
31     border:none;
32     padding:5px 8px;
33     cursor: pointer;
34   }
35 </style>
36 </head>
37 <body>
38 <h2>Task Manager</h2>
39 <input type="text" id="taskInput" placeholder="Enter task">
40 <button onclick="addTaskAsync()">Add Task</button>
41 <ul id="taskList"></ul>
42
43 <script>
44 let tasks = [];
45
46 const addTaskPromise = (task) => {
47   return new Promise((resolve) => {
48     setTimeout(() => {
49       tasks.push(task);
50       resolve();
51     }, 1000);
52   });
53 };
54
```

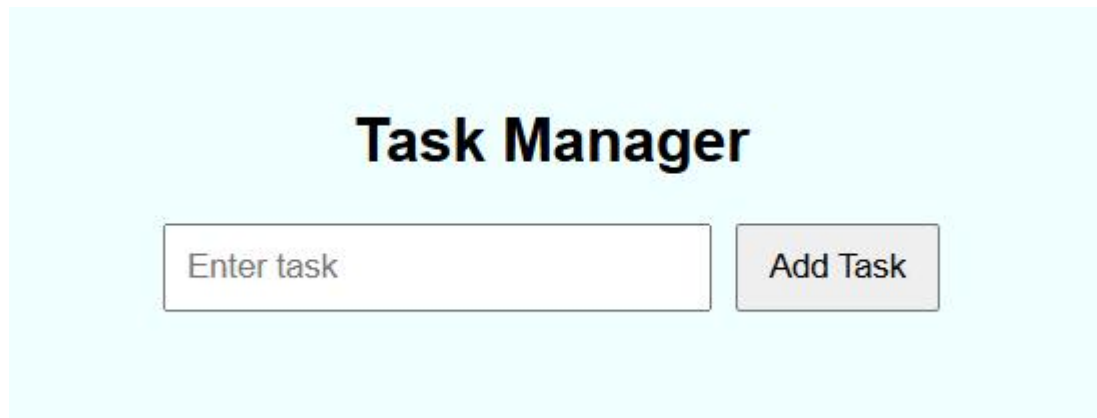
```

55 const deleteTaskPromise = (index) => {
56   return new Promise((resolve) => {
57     setTimeout(() => {
58       tasks.splice(index, 1);
59       resolve();
60     }, 1000);
61   });
62 };
63
64 const listTasksPromise = () => {
65   return new Promise((resolve) => {
66     setTimeout(() => {
67       resolve(tasks);
68     }, 1000);
69   });
70 };
71
72 const addTaskAsync = async () => {
73   const input = document.getElementById("taskInput");
74   const task = input.value;
75   if(task === "") {
76     alert("Enter a task");
77     return;
78   }
79   await addTaskPromise(task);
80   input.value="";
81   displayTasks();
82 };
83
84 const deleteTaskAsync = async (index) => {
85   await deleteTaskPromise(index);
86   displayTasks();
87 };
88
89 const displayTasks = async () => {
90   const taskList = document.getElementById("taskList");
91   const tasksData = await listTasksPromise();
92   taskList.innerHTML="";
93   tasksData.forEach((task, index) => {
94     taskList.innerHTML += `
95       <li>
96         ${task}
97         <button class="deleteBtn" onclick="deleteTaskAsync(${index})">Delete</button>
98       </li>
99     `;
100   });
101 };
102 </script>
103 </body>
104 </html>

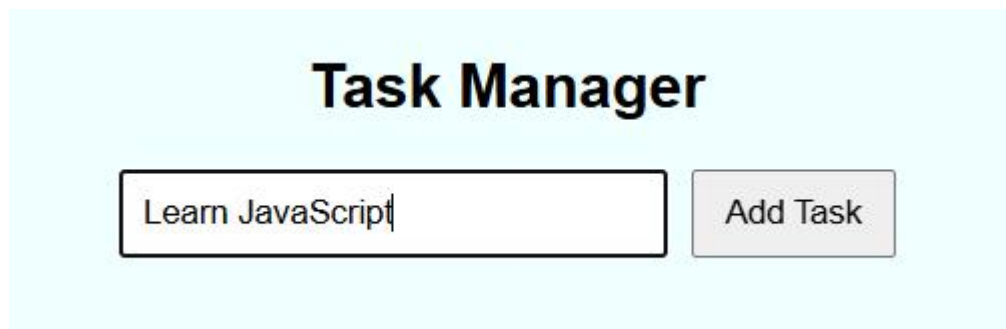
```

Output:

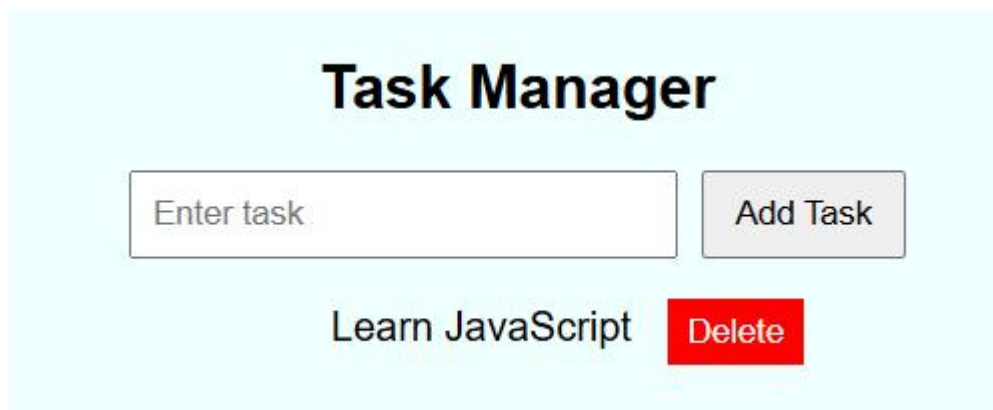
Output File: index.html



A screenshot of a web application titled "Task Manager" on a light blue background. Below the title is a white input field with the placeholder text "Enter task" and a grey "Add Task" button to its right.



A screenshot of the "Task Manager" web application. The input field now contains the text "Learn JavaScript" and the "Add Task" button is still present to its right.



A screenshot of the "Task Manager" web application showing the task list. The input field contains "Enter task" and the "Add Task" button is to its right. Below the input field, the text "Learn JavaScript" is displayed next to a red "Delete" button.

Explanation:

This code creates a simple **Task Manager** where a user can type a task and click **Add Task** to add it to the list. The tasks are stored in an array and displayed on the page, and each task has a **Delete button** to remove it. It uses **Promises and async/await with setTimeout** to simulate asynchronous operations like adding, listing, and deleting tasks before updating the UI.